



Software Security

- Lecture By
Pratidnya S. Hegde Patil
Assistant Professor-IT Dept.



Topics

- Categories of software implementation vulnerabilities,
- Techniques that can be used to Prevent such vulnerabilities,
- Detection and,
- Mitigation of the exploitation.

Courtesy: CyBOK, The Cyber Security Body of Knowledge, Chapter 14, <https://www.cybok.org/>



What it means for software to be secure?

- One possible definition is that a **software system is secure if it satisfies a specified or implied security objective**. This security objective specifies **confidentiality, integrity and availability** requirements for the system's data and functionality.
- Consider, for instance, **a social networking service**. The security objective of such a system could include the following requirements:
 - Pictures posted by a user can only be seen by that user's friends (**confidentiality**)
 - A user can like any given post at most once (**integrity**)
 - The service is operational more than 99.9% of the time on average (**availability**)

- 
- **Security Failure:** is a scenario where the software system does not achieve its security objective.
 - **Vulnerability:** is the underlying cause of such a failure.
 - Existing classifications of vulnerabilities, such as the **Common Weakness Enumeration (CWE)**, a community-developed list of vulnerability categories, are useful as a baseline for vulnerability identification, mitigation and prevention, but none of the existing classifications have succeeded in coming up with a complete taxonomy.



Categories of Vulnerabilities

1. **Memory Management Vulnerabilities**
2. Structured Output Generation Vulnerabilities
3. Race Condition Vulnerabilities
4. API Vulnerabilities
5. Side-channel Vulnerabilities



1. Memory Management Vulnerabilities

- C-like languages offer mutable state that can be allocated, deallocated and accessed in a number of ways:
 - Local and global variables, malloc() / free(), ...
 - Access through pointers and array indexing
- These memory management and access operations should be used correctly, e.g.:
 - Access arrays within bounds
 - Do not access memory after it has been deallocated
- For performance, compilers do not detect invalid memory accesses
 - Instead, behavior of the program becomes undefined
 - A program that can perform such an invalid access has a memory management vulnerability

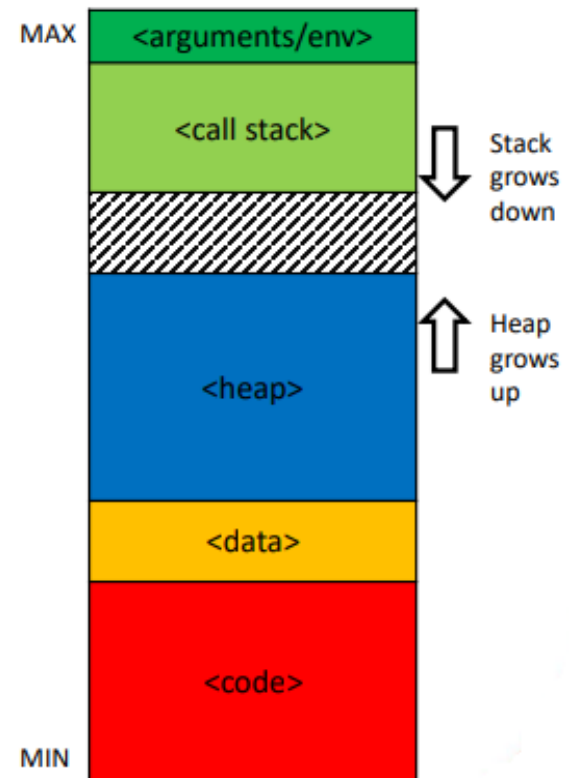


1. Memory Management Vulnerabilities

- Memory management vulnerabilities are particularly dangerous from a security point of view, because in many implementations **mutable memory cells allocated to the program are part of the same memory address space where also compiled program code, and runtime metadata such as the call stack are stored.**
- In such implementations, a memory access by the program that violates the memory management contract can result in an access to compiled program code or runtime metadata, and hence can cause corruption of program code, program control flow and program data.

1. Memory Management Vulnerabilities

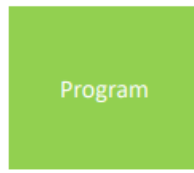
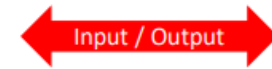
- Each function is compiled separately and the resulting machine code is stored in a **code** section in memory
- Control-flow through the program is tracked by means of a **call stack**
- Variables used in the program are allocated in a number of ways:
 - Local variables are allocated on the call stack
 - Global variables are allocated in a dedicated **data** section in memory
 - Dynamic allocation is handled by a memory management library that manages a **heap**
 - malloc(), free(), ...
- Pointers are represented as integer addresses, supporting pointer arithmetic
- Arrays are represented as pointers, indexing is similar to pointer arithmetic





Source of Security bugs in memory unsafe languages (c, c++)

- A **spatial vulnerability** is a bug where the program is indexing into a valid contiguous range of memory cells, but the index is out-of-bounds. The archetypical example is a **buffer overflow vulnerability** where the program accesses an array (a buffer) with an out-of-bounds index.
- A **temporal vulnerability** is a bug where the program accesses memory that was once allocated to the program, but has since been deallocated. A typical example is dereferencing a dangling pointer.



1. Memory Management Vulnerabilities

- **Attacker Model** : providing input to the program to trigger the vulnerability, which makes the program violate the memory management contract.
 - We consider attacks that consist of crafting malicious input and learning from output of the program
 - Attacker knows the code and system software stack of the victim
 - Attacker can send arbitrary input and inspect resulting output



1. Memory Management Vulnerabilities

Attacker chooses input to access a memory cell of interest to the attacker

- In a **code corruption attack**, the invalid memory access modifies compiled program code to attacker specified code.
- In a **control-flow hijack attack**, the invalid memory access modifies a code pointer (for instance, a return address on the stack, or a function pointer) to make the processor execute attacker-provided code (a direct code injection attack), or to make the processor reuse existing code of the program in unexpected ways (a code-reuse attack, also known as an indirect code injection attack, such as a return-to-libc attack, or a return oriented-programming attack).



1. Memory Management Vulnerabilities

Attacker chooses input to access a memory cell of interest to the attacker

- In a **data-only attack**, the invalid memory access modifies other data variables of the program, possibly resulting in increased privileges for the attacker.
- In an **information leak attack**, the invalid memory access is a read access, possibly resulting in the exfiltration of information, either application secrets such as cryptographic keys, or runtime metadata such as addresses which assist prediction of the exact layout of memory and hence may enable other attacks.



Categories of Vulnerabilities

1. Memory Management Vulnerabilities
2. Structured Output Generation Vulnerabilities
3. Race Condition Vulnerabilities
4. API Vulnerabilities
5. Side-channel Vulnerabilities



2. Structured output generation vulnerabilities

- Programs often have to dynamically construct structured output that will then be consumed by another program. One can think of the code that generates the structured output as a **sub-component**. The intended structure of the output, and how input to the sub-component should be used within the output, can be thought of as a **contract to which that sub-component should adhere**.
- Examples include: the construction of SQL queries to be consumed by a database, or the construction of HTML pages to be consumed by a web browser.



2. Structured output generation vulnerabilities (aka: injection vulnerabilities)

- Programs often construct structured output (eg. SQL) using string concatenation
- When some of the strings can be chosen by an attacker, maliciously chosen values can change the structure of the output in unintended ways
- Ex: SQL injection, script injection (XSS), command injection,....

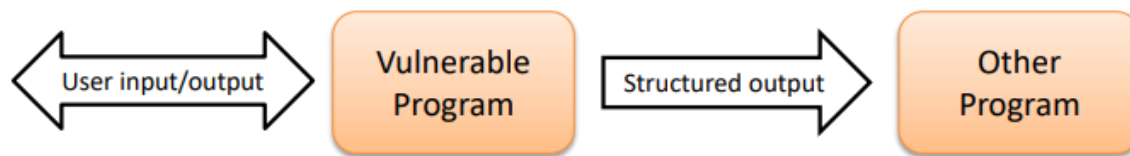


2. Structured output generation vulnerabilities (aka: injection vulnerabilities)

- A **SQL injection** vulnerability is a structured output generation vulnerability where the structured output consists of SQL code. These vulnerabilities are particularly relevant for server-side web application software, where it is common for the application to interact with a back-end database by constructing queries partially based on input provided through web forms.
- A **command injection** vulnerability is a structured output generation vulnerability where the structured output is a shell command sent by the application to the operating system shell.
- A **script injection** vulnerability, sometimes also called a Cross-Site Scripting (XSS) vulnerability is a structured output generation vulnerability where the structured output is JavaScript code sent to a web browser for client-side execution.

2. Structured output generation vulnerabilities (aka: injection vulnerabilities)

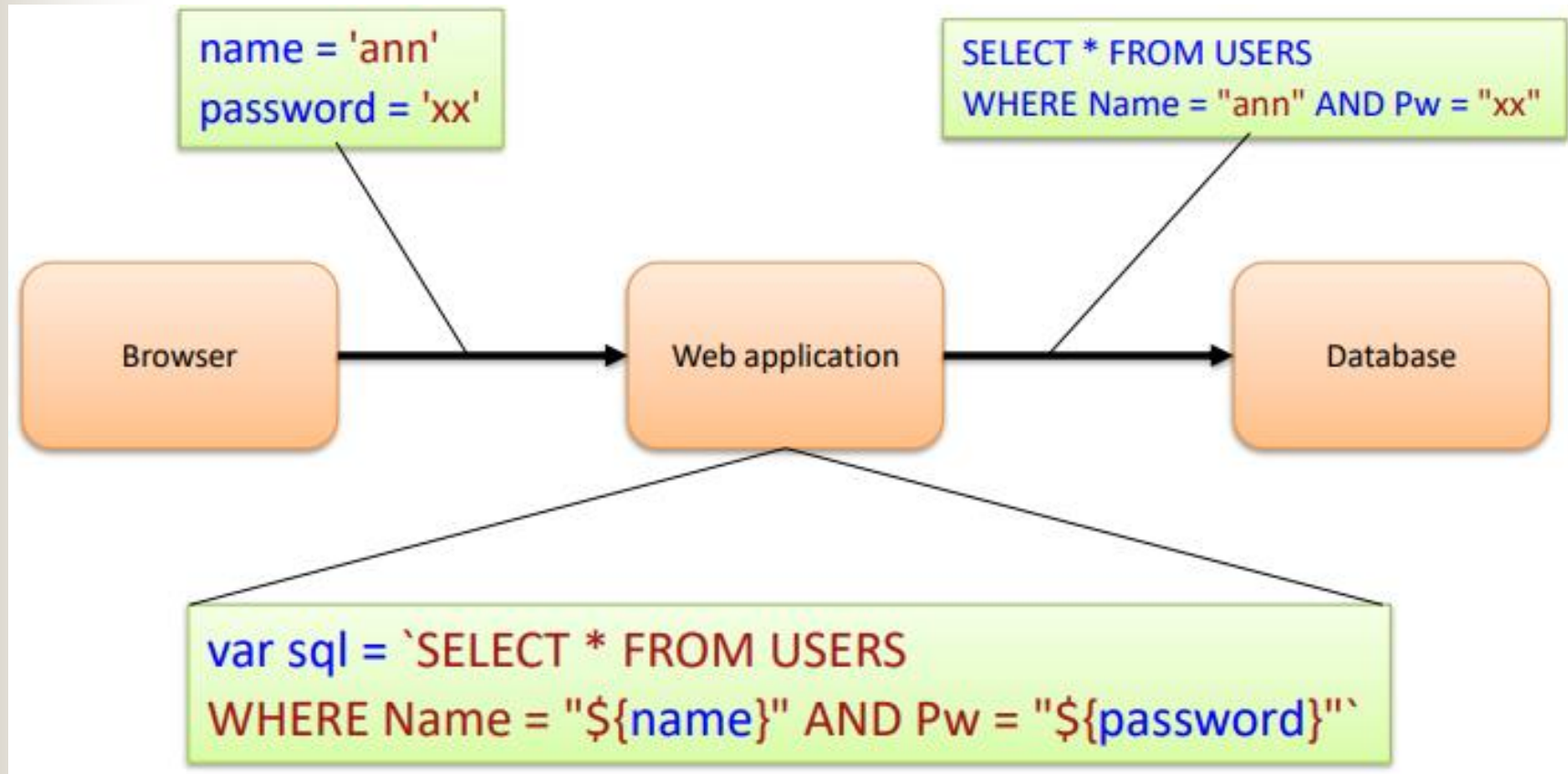
■ System Model



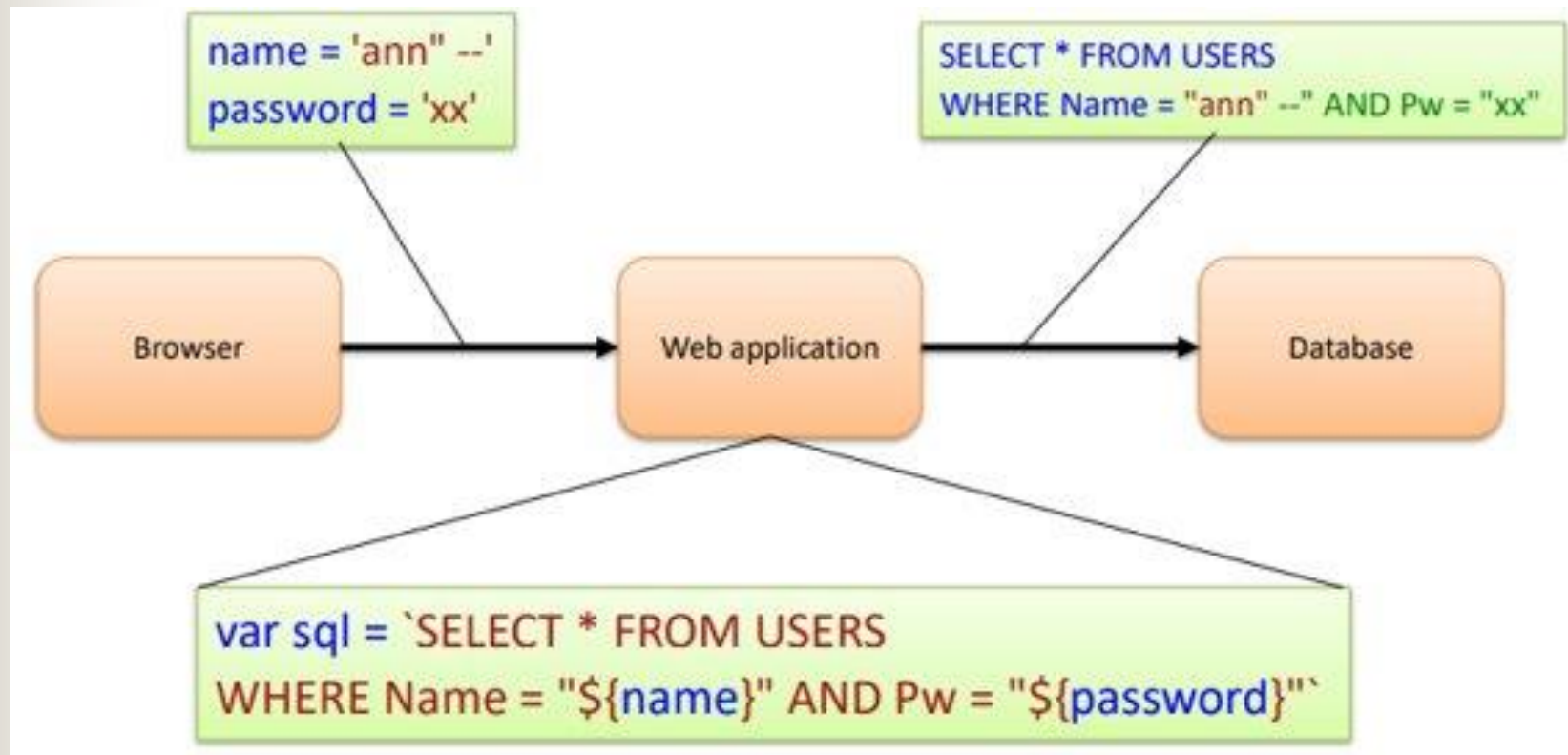
■ Attacker Model

- Attacker knows the code of the system
- Attacker can provide user input and read user output

SQL injection attack



SQL injection attack





2. Structured output generation vulnerabilities (aka: injection vulnerabilities)

- Structured output generation vulnerabilities may seem conceptually simple, but several factors can contribute to the difficulty of avoiding them:
 - Structured output in languages that have sublanguages with a different structure, e.g. HTML with JavaScript, CSS, SVG, ...
 - Structured output generation can happen in different phases, where output of one phase is later used as input for the next phase
 - Stored injection vulnerabilities
 - Higher-order injection vulnerabilities



Categories of Vulnerabilities

1. Memory Management Vulnerabilities
2. Structured Output Generation Vulnerabilities
3. Race Condition Vulnerabilities
4. API Vulnerabilities
5. Side-channel Vulnerabilities



3. Race condition vulnerabilities

- When a program accesses **resources (such as memory, files or databases)** that it shares with other **concurrent actors (other threads in the same process, or other processes)**, the program often makes assumptions about what these concurrent actors will do (or not do) to these shared resources.
- Concurrency bugs, also commonly referred to as race conditions, because a consequence of these bugs is that the behaviour of the program may depend on **which concurrent actor accesses a resource first ('wins a race')**.
- Ex: the program checks a condition on a resource, and then relies on that condition when using the resource. If an attacker can interleave his/her own actions to invalidate the condition between the check and the time of use, this is called a **Time Of Check Time Of Use (TOCTOU) vulnerability**.



3. Race condition occurs in

- Race conditions on the **file system**: privileged programs (i.e., programs that run with more privileges than their callers, for instance, operating system services) often need to check some condition on a file, before performing an action on that file on behalf of a less privileged user. Failing to perform check and action atomically (such that no con-current actor can intervene) is a race condition vulnerability: an attacker can invalidate the condition between the check and the action.
- Races on the **session state in web applications**: web servers are often multi-threaded for performance purposes, and consecutive HTTP requests may be handled by different threads. Hence, two HTTP requests belonging to the same HTTP session may access the session state concurrently. Failing to account for this is a race condition vulnerability that may lead to corruption of the session state.



Categories of Vulnerabilities

1. Memory Management Vulnerabilities
2. Structured Output Generation Vulnerabilities
3. Race Condition Vulnerabilities
4. **API Vulnerabilities**
5. Side-channel Vulnerabilities



4. API Vulnerabilities

- An Application Programming Interface, or API, is the interface through which one software component communicates with another component, such as a software library, operating system, web service, and so forth.



4. API Vulnerabilities

■ Attacker Model

- An API comes with an (explicit or implicit) specification/contract of how it should be used and what services it offers, and just like the contracts we considered in previous subsections, violations of these contracts can often have significant consequences for security.
- If the client of the API violates the contract, the software system again enters an error-state, and the further behaviour of the software system will depend on implementation details of the API, and this may allow an attacker to break the security objective of the overall software system.



Categories of Vulnerabilities

1. Memory Management Vulnerabilities
2. Structured Output Generation Vulnerabilities
3. Race Condition Vulnerabilities
4. API Vulnerabilities
5. Side-channel Vulnerabilities



5. Side-channel vulnerabilities

- A side-channel is an information channel leaking information about the execution of a program through effects not intended for communication
 - E.g. timing, power consumption, electro-magnetic radiation, ...
- Many side-channels require physical access to the system running the program under attack.
- Software-based side-channels can be read by software running on the same system as the program under attack
- A very recent important class of such channels exploit effects of the processor implementation. – Software-based micro-architectural side-channels

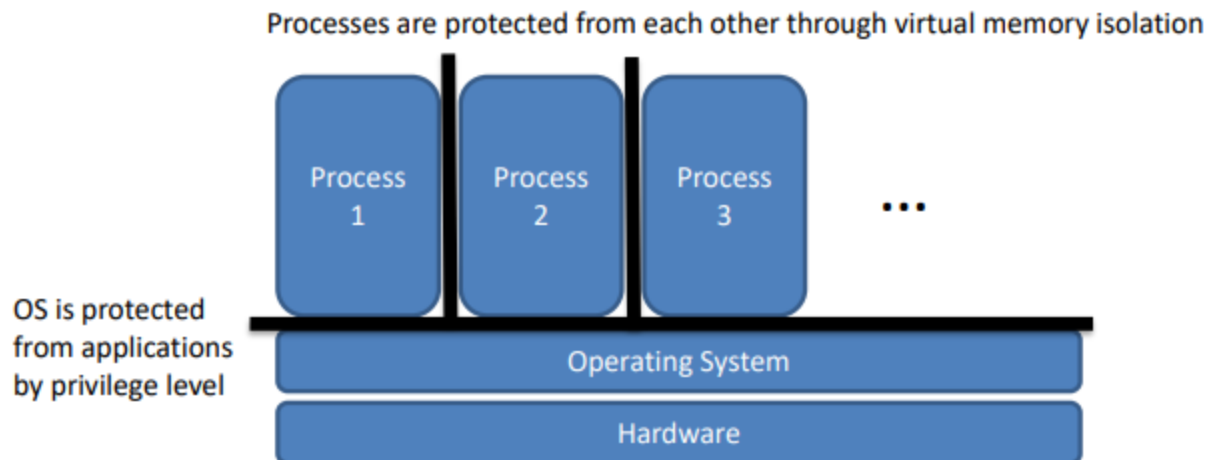
- 
- Side-channels play an important role in the field of **cryptography**, where the abstraction gap between
 - (1) the mathematical (or source code level) description of a cryptographic algorithm and
 - (2) the physical implementation of that algorithm, has been shown to be relevant for security.
 - It was demonstrated that, unless an implementation carefully guards against this, side-channels based on **power consumption or execution time** can easily **leak the cryptographic key** used during the execution of an encryption algorithm. This breaks the security objectives of encryption for an attacker model where the attacker can physically monitor the encryption process.

5. Side-channel vulnerabilities

System model: a shared platform

A platform runs programs from multiple stakeholders

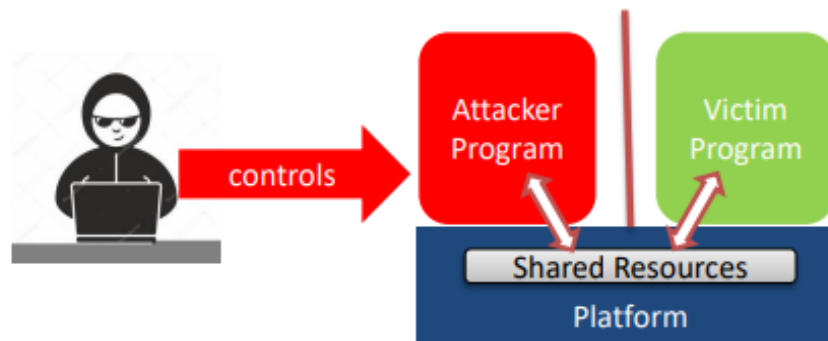
- Isolation mechanism isolates these programs
- The platform supports communication between these programs



5. Side-channel vulnerabilities

Attack model

- The attacker can run code on the same platform where victim code is running.
- The objective of the attacker is to learn more about the victim than what one can learn through intended communication interfaces.





Prevention of Vulnerabilities

- Language Design and Type Systems
 - Memory Management Vulnerabilities
 - Structured Output Generation Vulnerabilities
 - Race Condition Vulnerabilities
 - Other Vulnerabilities
- API Design
- Coding Practices



Memory Management Vulnerabilities

- A language can be made memory-safe through a combination of:
 1. the careful selection of the features it supports: for instance, languages can choose to avoid mutable state, or can choose to avoid dynamic memory allocation, or can choose to avoid manual deallocation by relying on garbage collection,
 2. imposing dynamic checks: for instance, imposing that every array access must be bounds-checked, and
 3. imposing static checks, typically in the form of a static type system: for instance, object field access can be guaranteed safe by means of a type system.



Structured Output Generation Vulnerabilities

- **A first approach:** is to provide a type system that supports the description of structured data. This approach has been worked out rigorously for XML data: the programming language supports XML documents as first class values, and regular expression types support the description of the structure of XML documents using the standard regular expression operators. A type-correct program that outputs an XML document of a given type is guaranteed to generate XML output of the structure described by the type.
- **A second approach:** is to provide primitive language features for some of the common use cases of structured output generation. Language Integrated Query (LINQ) is an extension of the C# language with syntax for writing query expressions. By writing the query as an expression (as opposed to building a SQL query by concatenating strings), the intended structure of the query is explicit, and the LINQ provider that compiles the query to SQL can provide strong guarantees that the generated query has the intended structure.



Race Condition Vulnerabilities

- Race condition vulnerabilities on heap allocated memory are often enabled by aliasing, the existence of multiple pointers to the same memory cell.
- The notion of ownership helps mitigate the complications that arise because of aliasing. The essence of the idea is that, while multiple aliases to a resource can exist, only one of these aliases is the owner of the resource, and some operations can only be performed through the owner.
- For instance, a simple ownership regime for heap allocated memory cells might impose the constraints that: (1) aliases can only be created if they are guaranteed to go out of scope before the owner does, (2) aliases can only be used for reading, and (3) the owner can write to a cell only if no aliases currently exist. This simple regime avoids data races: there can never be a concurrent read and write on the same cell. It also supports automatic memory management without garbage collection: a heap cell can be deallocated as soon as the owner goes out of scope.



API Design

- API should be designed to avoid execution errors which are violations of the API specification.
- It should be difficult for the programmer to violate an API contract, and if the contract is violated, that should be trapped, leading, for instance, to program termination or to well-defined error-handling behaviour.
- Assertions, contracts and defensive programming are approaches to avoid API vulnerabilities. Design by contract makes the contract of an API explicit by providing preconditions and postconditions, and in defensive programming these preconditions will be checked, thus avoiding the occurrence of untrapped errors.



API Design

- Several libraries providing less error-prone APIs for memory management in C or C++ have been proposed. These libraries offer fat pointers (where pointers maintain bounds information and check whether accesses are in bound), garbage collection (where manual deallocation is no longer required), or smart pointers (that support an ownership regime to safely automate deallocation).
- Several libraries providing less error-prone APIs to do structured output generation for various types of structured output and for various programming languages have been proposed. Examples include Prepared Statement APIs that allow a programmer to separate the structure of a SQL statement from the user input that needs to be plugged into that structure, or library implementations of language integrated query, where query expressions are constructed using API calls instead of using language syntax.
- Several libraries providing less error-prone APIs to cryptography have been proposed. These libraries use simplification (at the cost of flexibility), secure defaults, better documentation and the implementation of more complete use-cases (for instance, include support for auxiliary tasks such as key storage) to make it less likely that a developer will make mistakes.



Coding Practices

- Secure coding practices can be formalised as collections of rules and recommendations that describe and illustrate good and bad code patterns.
- Rules can take many forms, including:
 - the avoidance of dangerous language provided API functions (e.g., do not use the `system()` function in C),
 - attempting to avoid undefined behaviour or untrapped execution errors (e.g., do not access freed memory in C),
 - mitigations against certain vulnerabilities caused by the language runtime (e.g., not storing secrets in Java Strings, as the Java runtime can keep those Strings stored on the heap indefinitely), or,
 - proactive, defensive rules that make it less likely to run into undefined behaviour (e.g., exclude user input from format strings).



Detection of Vulnerabilities

- Techniques must make a trade-off between:
- Sound: if it can correctly conclude that a given program has no vulnerabilities of that category. Not lead to false negatives (i.e. actual vulnerabilities that the detection technique fails to find).
- Complete: if any vulnerability it finds is an actual vulnerability. Not lead to false positives (i.e. detect issues that do not turn out to be actual vulnerabilities).



Detection of Vulnerabilities

- Achieving soundness can be done by static checking of the program code while making suitable abstractions of the executions to make the analysis terminate.
- Achieving completeness can be done by dynamic detection where the analysis technique has to come up with concrete inputs for the program that trigger a vulnerability.



Detection of Vulnerabilities

- Static Detection: Analyse source code or binary code. Have advantage as can operate on incomplete code that is not yet executable.
 - Heuristic static detection
 - Sound static verification
- Dynamic Detection: Execute a program and monitor the execution to detect vulnerabilities.
 - **Monitoring:** specific vulnerability checks.
 - **Generating relevant executions:** systematically selecting appropriate inputs for a program under test.



Static Detection

- **Heuristic static detection:**

- The static analysis technique builds a semantic model of the program, including, for instance, an abstract syntax tree, and abstractions of the data flow and control flow in the program. Based on this model, the technique can flag violations of simple syntactic rules such as, do not use this dangerous API function, or only use this API function with a constant string as first parameter.

- **Sound static verification:**

- Program verification:
- Abstract interpretation:
- Model Checking:



Dynamic detection

- **Monitoring:**
 - Monitoring for memory-management vulnerabilities
 - Monitoring for structured output generation vulnerabilities
 - Monitoring API vulnerabilities
 - Monitoring for race conditions



Dynamic detection

- **Generating relevant executions:**
 - Black-box fuzzing: Does not depend on internal structure but only on I/O behaviour of the program being tested.
 - Random-based
 - Model-based
 - Mutation-based
 - White-box fuzzing: The internal structure of the program is analysed to assist in the generation of appropriate input values. By solving for input values that do not satisfy the path condition of the current execution, the fuzzer can make sure that these input values will drive the program to a different execution path, thus improving coverage.



Mitigating Exploitation of Vulnerabilities

- Runtime Detection of Attacks
- Automated Software Diversity
- Limiting Privileges
- Software Integrity Checking



Mitigation

- **Runtime Detection of Attacks (Structured output generation vulnerabilities, Memory Management vulnerabilities)**
 - Detect vulnerabilities during testing. On detection of an attack, the runtime monitor must react appropriately, usually by terminating the program under attack. Termination is a good reaction to ensure that an attack can do no further damage, but it has of course a negative impact on availability properties.
 - Stack canaries detect violations of the integrity of activation records on the call stack, and hence detect some attacks that exploit memory management vulnerabilities to modify a return address.
 - Non-executable data memory (NX) detects attempts to direct the program counter to data memory instead of code memory and hence detects many direct code injection attacks.
 - Control Flow Integrity (CFI) is a class of techniques that monitors whether the runtime control flow of the program complies with some specification of the expected control flow, and hence detects many code-reuse attacks.



Mitigation

■ Automated Software Diversity

- Exploitation of vulnerabilities often relies on implementation details of the software under attack. For instance, exploitation of a memory management vulnerability usually relies on details of the memory layout of the program at runtime. A SQL injection attack can rely on details of the database to which the SQL query is being sent.
- A generic countermeasure to make it harder to exploit vulnerabilities is to diversify these implementation details.
 - Harder for an attacker to prepare and test his/her attack on an identical system.
 - Harder for an attacker to build attacks that will work against many systems at once.



Mitigation

- **Limiting Privileges (side channel vulnerabilities, race condition vulnerabilities, API vulnerabilities, Memory management vulnerabilities)**
 - **Sandboxing:** software is executed within a controlled environment (the ‘sandbox’) and where a policy can be enforced on the resources that software in the sandbox can access. Sandboxing can be used to confine untrusted software, but it can also be used to mitigate the impact of exploitation on vulnerable software: after a successful exploit on the software, an attacker is still confined by the sandbox.
 - **Compartmentalisation:** finer-grained security mechanism, where the software itself is divided in a number of compartments and where some bounds are enforced on the privileges of each of these compartments.
 - To mitigate side-channel vulnerabilities, one can isolate the vulnerable code, for instance, on a separate core or on separate hardware, such that the information leaking through the side channel is no longer observable for attackers.



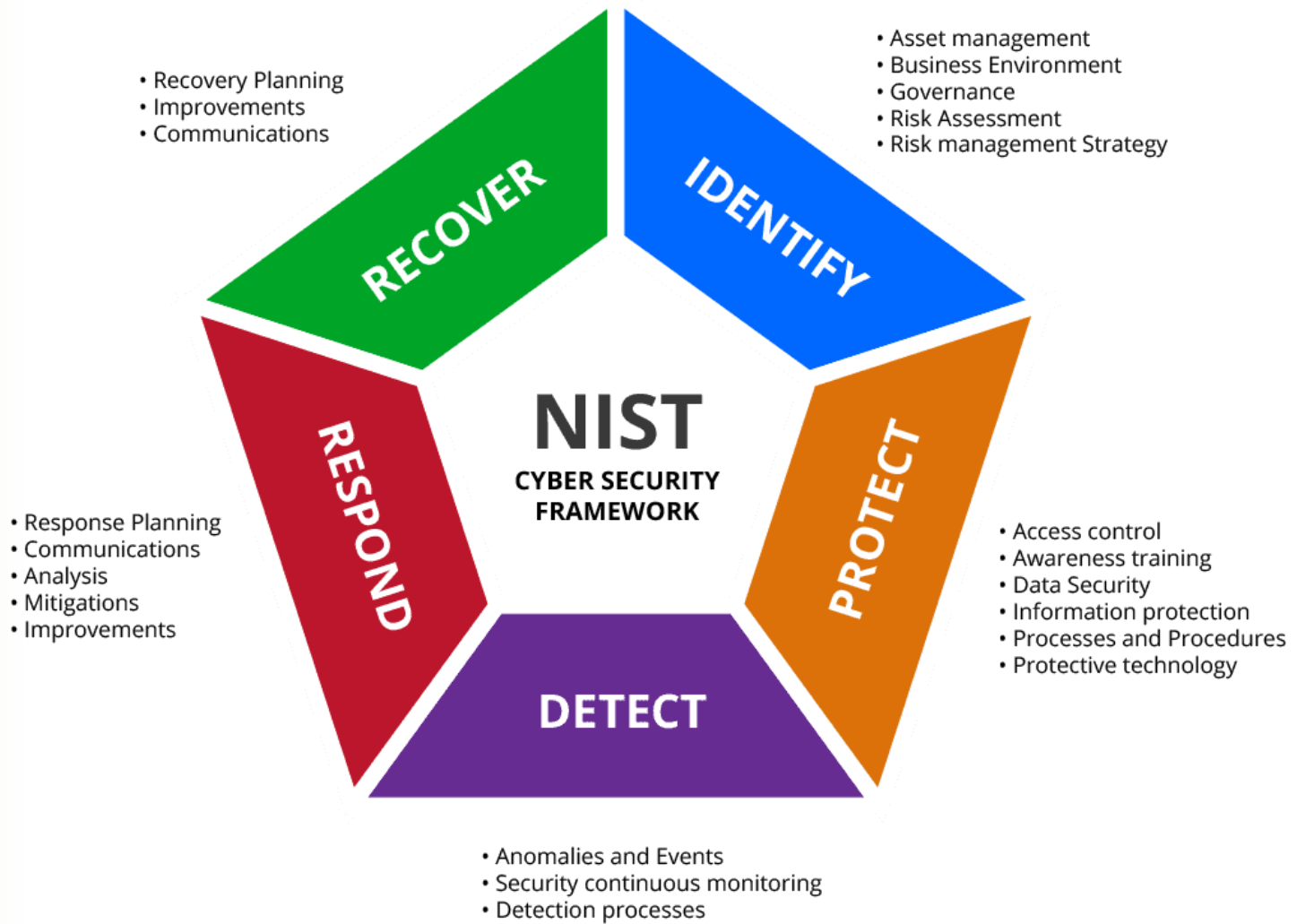
Mitigation

■ Software Integrity Checking

- Trusted Boot where measurements are accumulated for each program that is executed. Any modification to the programs (for instance, because of a successful attack) will lead to a different measurement. One can then enforce that access to secret keys, for instance, is only possible from a state with a specified measurement.

Summary

Vulnerability category	Prevention	Detection	Mitigation
Memory management vulnerabilities	memory-safe languages, fat/smart pointers, coding rules	many static and dynamic detection techniques	stack canaries, NX, CFI, ASLR, sandboxing
Structured output generation vulnerabilities	regular expression types, LINQ, Prepared Statements	taint analysis	runtime detection
Race condition vulnerabilities	ownership types, coding guidelines	static and dynamic detection	sandboxing
API vulnerabilities	contracts, usable APIs, defensive API implementations	runtime checking of pre- and post-conditions, static contract verification	compartmentalisation
Side channel vulnerabilities	coding guidelines	static detection	isolation





NIST

- The National Institute of Standards and Technology is a non-regulatory government agency that develops technology, metrics, and standards to drive innovation and economic competitiveness at U.S.-based organizations in the science and technology industry. As part of this effort, NIST produces standards and guidelines to help federal agencies meet the requirements of the Federal Information Security Management Act (FISMA). NIST also assists those agencies in protecting their information and information systems through cost-effective programs.



Applicability

- NIST is a federal agency within the United States Department of Commerce. NIST's mission is to develop and promote measurement, standards, and technology to enhance productivity, facilitate trade, and improve the quality of life. NIST is also responsible for establishing computer- and information technology-related standards and guidelines for federal agencies to use. Many private sector organizations have made widespread use of these standards and guidelines voluntarily for several decades, especially those related to information security.



Objective

- The NIST CSF uses these 5 core areas to evaluate security controls:
 - Identify
 - Protect
 - Detect
 - Respond
 - Recover
- These five areas represent the lifecycle of cybersecurity risk. Each area has categories tied to specific needs and activities. Each category is further broken down into subcategories, standards, guidelines, and practices which are needed to accomplish the outcome for that category.



Approach

Our approach has been covered in phases as mentioned below:

- **Phase 1: Understand Business Process:** Understanding the environment and management's expectations along with the policies and procedures.
- **Phase 2: Identify Risks and Controls:** Identify target processes and understand the process flow, risk, information assets and controls pertaining to processes.
- **Phase 3: Controls Design Testing:** Identify controls based of NIST and prepare the issue and opportunity registers, test the control design and identify deficiencies. Prepare risk mitigation plan and calculate the residual risks.
- **Phase 4: Controls Evaluation :** Perform internal audit and identify the control weaknesses and impact of deficiencies.
- **Phase 5: Reporting**